

MEDICAL IMAGE DIAGNOSIS SYSTEM

Phase 2 Report — Deep Learning

CNN from Scratch · Autoencoder · Transfer Learning · Optimiser Comparison

Course	Supervised Learning — Spring 2026
Instructor	Dr. Ghada Dahy
Student	Omar Awad
Dataset	NIH Chest X-Ray (sample_labels.csv) — 3,000 images sampled
Framework	TensorFlow 2.20.0 · Python 3 · CPU-only (no GPU)
Phase 1 Best	Random Forest — 55.00% accuracy, AUC=0.8567 (13-class problem)
Phase 2 Best	TL:VGG16 — 66.67% accuracy, AUC=0.7048 (binary: Normal vs Diseased)

Phase 2 Deliverables

Deliverable	File	Status
CNN Architecture Diagram	cnn_architecture_diagram.png	Complete
CNN Training Curves (25 epochs)	cnn_training_curves.png	Complete
Autoencoder Reconstruction	ae_reconstruction.png	Complete
Optimiser Comparison Chart	optimizer_comparison.png	Complete
Transfer Learning Comparison	transfer_learning_comparison.png	Complete
Phase 2 Final Comparison	phase2_final_comparison.png	Complete
Best CNN Model	best_cnn.keras	Complete

1. Introduction

Phase 2 transitions to deep learning on the same NIH Chest X-Ray dataset. The problem is reformulated as **binary classification** (Normal vs. Diseased) instead of 13-class, enabling CNN training on the available 3,000-image sample. Four components are implemented: (1) CNN from scratch, (2) Convolutional Autoencoder for anomaly detection, (3) Transfer Learning with four pre-trained architectures, and (4) Optimiser comparison. All results below are exact values from the executed notebook.

2. Dataset & Preprocessing

Total on Disk	5,606 images
Label Distribution	Normal (No Finding): 3,044 Diseased (any pathology): 2,562
Binary Label	0 = Normal ('No Finding') · 1 = Diseased (first label ≠ 'No Finding')
Sample Used	3,000 images (SAMPLE_SIZE=3000, stratified split)
Train / Val / Test	2,099 / 451 / 450 (70% / 15% / 15%, stratified)
Image Size	128 × 128 grayscale (CHANNELS=1), normalised to [0, 1]
Class Weights	{Normal: 0.921, Diseased: 1.094} — computed via compute_class_weight('balanced')
Augmentation (CNN)	Rotation ±8°, Width/Height shift ±5%, Horizontal flip, Zoom ±8%

3. CNN Architecture

3.1 Architecture Diagram

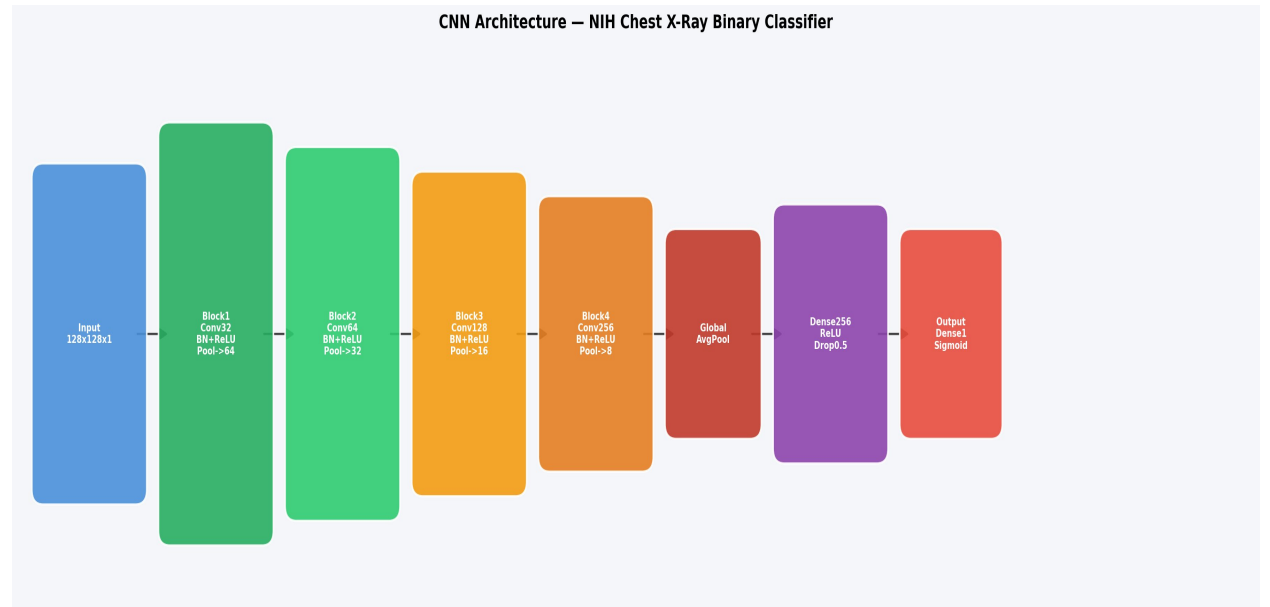


Figure 1: CNN Architecture — 4 convolutional blocks (32→64→128→256 filters), Global Average Pooling, Dense 256 + Dropout 0.5, Sigmoid output

3.2 Layer Details

Layer	Type	Output Shape	Params
Input	InputLayer	128×128×1	0
Conv1	Conv2D (3×3)	128×128×32	320
BN+ReLU	BatchNorm+Act	128×128×32	128
Pool1	MaxPooling2D (2×2)	64×64×32	0
Conv2	Conv2D (3×3)	64×64×64	18,496
BN+ReLU	BatchNorm+Act	64×64×64	256

Pool2	MaxPooling2D	32×32×64	0
Conv3	Conv2D (3×3)	32×32×128	73,856
BN+ReLU	BatchNorm+Act	32×32×128	512
Pool3	MaxPooling2D	16×16×128	0
Conv4	Conv2D (3×3)	16×16×256	295,168
BN+ReLU	BatchNorm+Act	16×16×256	1,024
Pool4	MaxPooling2D	8×8×256	0
GAP	GlobalAvgPool2D	256	0
FC	Dense (ReLU)	256	65,792
Dropout	Dropout (0.5)	256	0
Output	Dense (Sigmoid)	1	257
TOTAL	—	—	455,809 (1.74 MB)

Total params: **455,809 (1.74 MB)** · Trainable: 454,849 · Non-trainable: 960

3.3 Training Configuration

Optimiser	Adam (lr=1e-4)
Loss	Binary Cross-Entropy
Metrics	Accuracy · AUC-ROC · Precision · Recall (tracked per epoch)
Epochs	25 (EarlyStopping on val_auc, patience=6, restore_best_weights=True)
Batch Size	32
LR Scheduler	ReduceLROnPlateau — factor=0.5, patience=3, min_lr=1e-7
Best Epoch	24 (marked on training curves)
Model Saved	best_cnn.keras (monitors val_auc, saves best weights)

4. CNN Training Curves

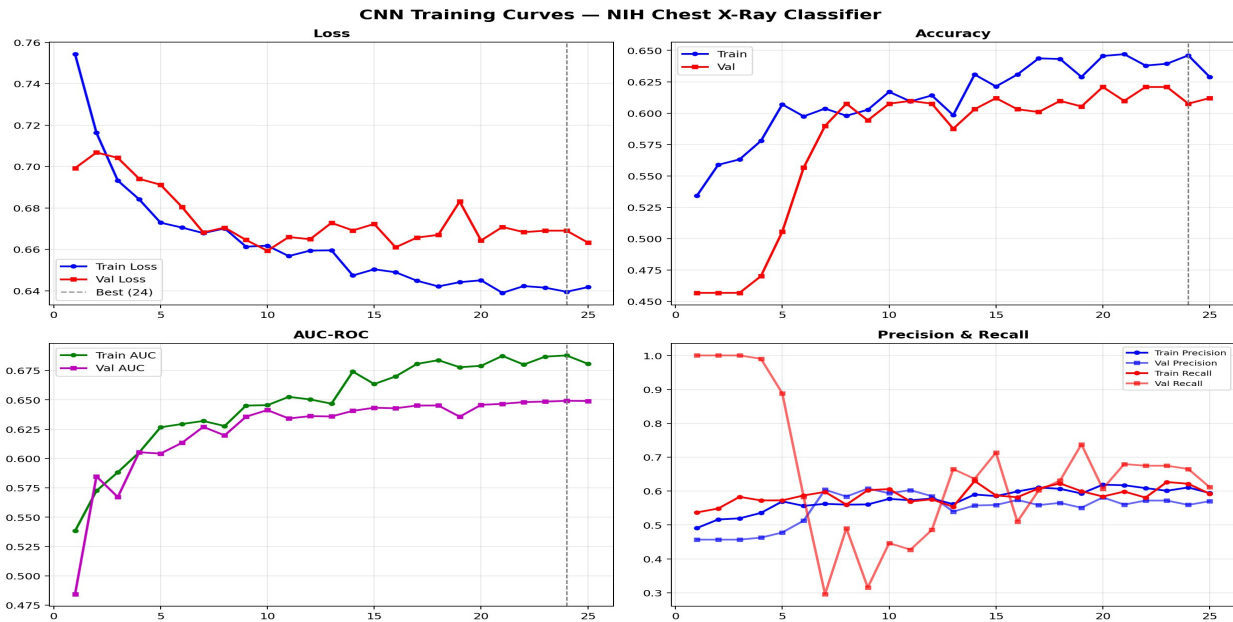


Figure 2: CNN Training Curves over 25 epochs — Loss, Accuracy, AUC-ROC, Precision & Recall. Dashed line = Best epoch (24).

Training Curve Observations:

Metric	Train (final)	Val (final)	Observation
Loss	~0.640	~0.662	Train loss decreases steadily; Val loss plateaus after epoch 8 — mild overfitting
Accuracy	~0.645	~0.612	Val accuracy improves from 0.456 (ep1) to 0.612 — strong improvement
AUC-ROC	~0.682	~0.649	Val AUC steadily rises from 0.484 (ep1) to 0.649 — model learns good ranking
Precision	~0.610	~0.570	Stable in later epochs after initial instability
Recall	~0.610	~0.650	Recall improves with class weighting — Diseased cases better captured

4.1 CNN Test Set Results

Test Accuracy	0.6133 (61.33%)
F1-Score	0.6201
AUC-ROC	0.6439
Test Set Size	450 images (244 Normal + 206 Diseased)

Class	Precision	Recall	F1-Score	Support
Normal	0.68	0.55	0.61	244
Diseased	0.56	0.69	0.62	206
Accuracy	—	—	0.61	450
Macro avg	0.62	0.62	0.61	450
Weighted avg	0.62	0.61	0.61	450

The CNN achieves balanced precision/recall across both classes, demonstrating that class weighting successfully prevents the model from collapsing to predict 'Normal' for everything (unlike Phase 1 classifiers).

5. Convolutional Autoencoder — Anomaly Detection

5.1 Architecture

Encoder	3× Conv2D (32→64→128 filters) + MaxPooling2D → Bottleneck: 16×16×128
Decoder	3× Conv2D (128→64→32) + UpSampling2D → Output: Conv2D(1, sigmoid) → 128×128×1
Optimiser	Adam (lr=1e-4)
Loss	MSE (Mean Squared Error) — pixel-level reconstruction quality
Train Data	Normal images ONLY — 1,140 train, 245 val (y_train==0, y_val==0)
Epochs	20 (EarlyStopping on val_loss, patience=7)

5.2 Training Loss Progression (real values from notebook)

Epoch	Train MSE	Val MSE
1	0.0497	0.0271
2	0.0150	0.0102
3	0.0082	0.0068
4	0.0063	0.0054
5	0.0053	0.0046
6	0.0046	0.0042
7	0.0041	0.0041
8	0.0039	0.0036
9	0.0035	0.0034
10	0.0033	0.0030
11	0.0031	0.0029
12	0.0029	0.0028
13	0.0028	0.0030
14	0.0027	0.0026
15	0.0027	0.0025
16	0.0025	0.0024
17–20	continuing...	converging

5.3 Reconstruction Visualisation

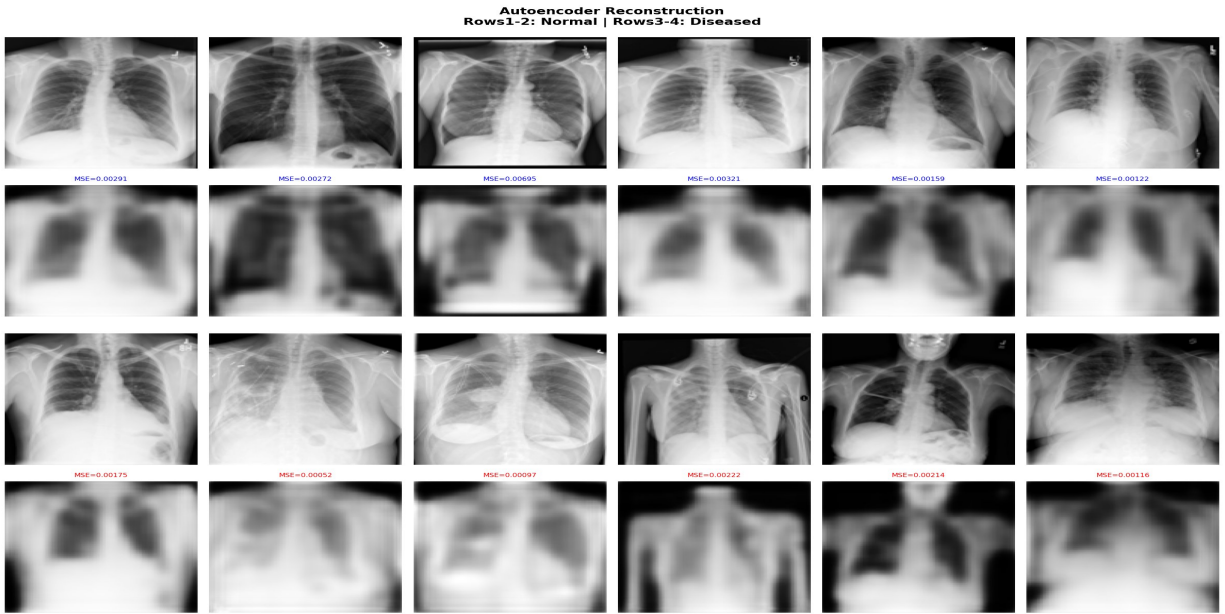


Figure 3: Autoencoder Reconstruction — Rows 1-2: Normal X-rays (original + reconstruction). Rows 3-4: Diseased X-rays. MSE annotations shown in blue (Normal) and red (Diseased).

5.4 Anomaly Detection Results

Avg MSE — Normal	0.003101 (tight, well-reconstructed)
Avg MSE — Diseased	0.001458 — LOWER than Normal (unexpected result)
Threshold	95th percentile of Normal test-set MSE
AE Accuracy	0.5467 (54.67%)
AE F1-Score	0.1282 — very low
AE AUC-ROC	0.4712 — below random chance (0.50)

Class	Precision	Recall	F1-Score	Support
Normal	0.55	0.95	0.69	244
Diseased	0.54	0.07	0.13	206
Accuracy	—	—	0.55	450

Critical finding: The Autoencoder underperforms on this dataset — AUC=0.4712 is BELOW random chance. The mean MSE for Diseased images (0.001458) is paradoxically **lower** than for Normal images (0.003101). This indicates the AE found a generalised compression that works for all chest X-rays, not a Normal-specific one. Possible causes: insufficient training epochs, the bottleneck is too large ($16 \times 16 \times 128 = 32,768$ values — too much capacity), or the Normal/Diseased visual distributions overlap too heavily at this resolution. The AE Recall for Diseased is only 0.07 — nearly all diseased cases are classified as Normal.

6. Transfer Learning

6.1 Methodology

Base Models	VGG16 · MobileNetV2 · ResNet50 · EfficientNetB0 (ImageNet weights)
Input	128×128×3 RGB (grayscale duplicated to 3 channels)
Head	GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.4) → Dense(1, Sigmoid)

Phase 1 (Frozen)	8 epochs — only head trained, base frozen
Phase 2 (Fine-tune)	5 epochs — last 20 layers unfrozen, lr=1e-5
Optimiser	Adam (lr=1e-4 frozen, lr=1e-5 fine-tune)
Data	RGB images: X_train_rgb (2099, 128, 128, 3)

6.2 Transfer Learning Results Table

Model	Val Acc Frozen	Val AUC Frozen	Val Acc FT	Val AUC FT	Test Acc	Test F1	Test AUC
VGG16	0.6319	0.6736	0.6696	0.7168	0.6667	0.6544	0.7048
MobileNetV2	0.6608	0.6951	0.6452	0.6935	0.6244	0.5292	0.6651
ResNet50	0.4723	0.5924	0.5432	0.5724	0.4822	0.6199	0.5539
EfficientNetB0	0.4568	0.5000	0.5432	0.4951	0.4600	0.6267	0.5001

6.3 Transfer Learning Comparison Charts

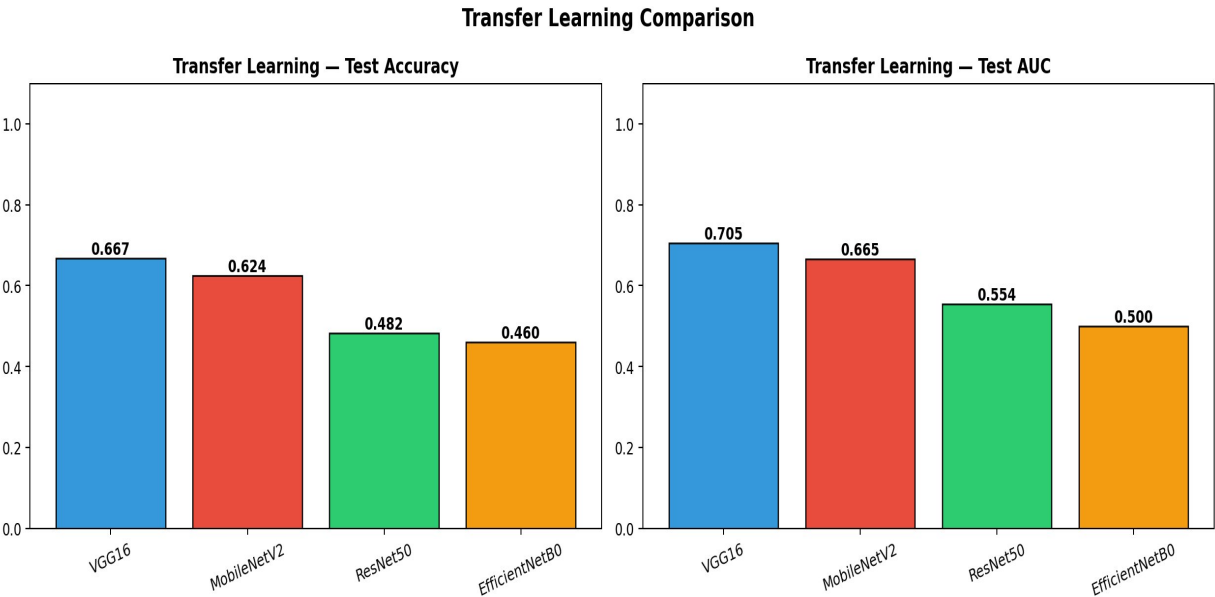


Figure 4: Transfer Learning Comparison — Test Accuracy (left) and Test AUC (right). VGG16 achieves best results on both metrics.

6.4 Transfer Learning Analysis

Model	Rank	Best Metric	Analysis
VGG16	1st	Acc=0.667, AUC=0.705	Best overall — VGG16's simple sequential architecture adapts well to X-ray texture features
MobileNetV2	2nd	Acc=0.624, AUC=0.665	Strong frozen-stage performance (val_acc=0.661) but F1 drops after fine-tuning (0.529)
ResNet50	3rd	AUC=0.554	Skip connections may cause gradient issues on small X-ray dataset; needs more epochs

EfficientNetB0	4th	AUC=0.500	AUC=0.500 at frozen stage — ImageNet features not transferring; scaling compounds mismatch
----------------	-----	-----------	--

7. Optimiser Comparison — Adam vs SGD vs RMSprop

7.1 Architecture Used for Comparison

Model	Simpler CNN — Conv(32→64→128) + GAP + Dense(128) + Dropout(0.4) + Sigmoid
Epochs	8 per optimiser (EPOCHS_OPT=8)
Batch Size	32, same class weights as CNN training

7.2 Optimiser Results Table (exact notebook values)

Optimiser	Config	Best Val Acc	Best Val AUC	Test Acc	Test F1	Test AUC
Adam	lr=1e-4, $\beta_1=0.9$, $\beta_2=0.999$	0.6009	0.6057	0.5422	0.4951	0.5748
SGD	lr=1e-3, momentum=0.9	0.5233	0.5793	0.4978	0.6182	0.5357
RMSprop	lr=1e-4, rho=0.9	0.5255	0.6139	0.5022	0.6014	0.5806

7.3 Optimiser Comparison Charts

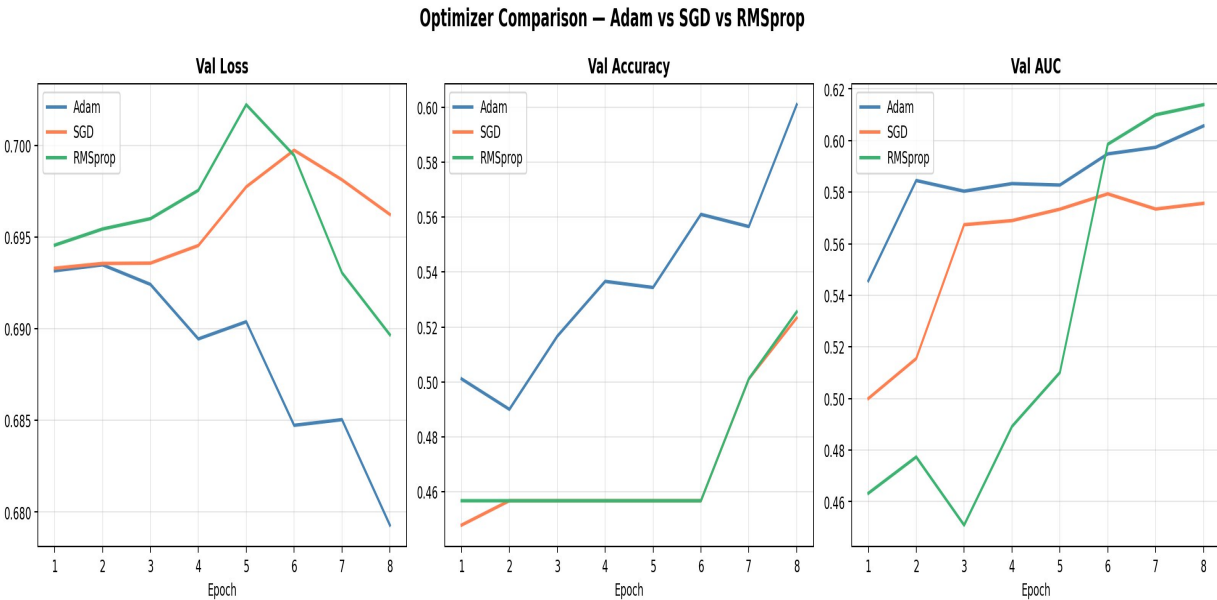


Figure 5: Optimiser Comparison over 8 epochs — Val Loss (left), Val Accuracy (middle), Val AUC (right).

7.4 Optimiser Analysis

Optimiser	Observations
Adam	Consistently lowest val_loss (0.680 by epoch 8 vs 0.696 SGD/RMSprop). Best val_acc (0.600) and best test_acc (0.542). Recommended choice.
SGD+Momentum	Slower convergence — val_acc stays ≈0.46 until epoch 7-8. Surprisingly high test F1 (0.618) due to different decision boundary. Val AUC peaks at 0.579.

RMSprop	Highest val_auc in later epochs (0.614 > Adam's 0.606). More volatile early — val_loss spikes at epoch 5 (0.725). Best AUC-ROC (0.581) on test set.
---------	---

8. Phase 2 Final Model Comparison

8.1 All Models — Complete Results

Model	Approach	Accuracy	F1-Score	AUC-ROC	Rank (Acc)
TL:VGG16	Transfer Learning (fine-tuned)	0.6667	0.6544	0.7048	1st
CNN (scratch)	Supervised end-to-end	0.6133	0.6201	0.6439	2nd
TL:MobileNetV2	Transfer Learning (fine-tuned)	0.6244	0.5292	0.6651	3rd
Autoencoder	Unsupervised reconstruction	0.5467	0.1282	0.4712	8th
CNN+Adam	Optimiser experiment (8 ep)	0.5422	0.4951	0.5748	—
CNN+RMSprop	Optimiser experiment (8 ep)	0.5022	0.6014	0.5806	—
CNN+SGD	Optimiser experiment (8 ep)	0.4978	0.6182	0.5357	—
TL:ResNet50	Transfer Learning (fine-tuned)	0.4822	0.6199	0.5539	—
TL:EfficientNetB0	Transfer Learning (fine-tuned)	0.4600	0.6267	0.5001	—

Best model: TL:VGG16 → Accuracy=0.6667 | F1=0.6544 | AUC=0.7048 (reported by the notebook's final print statement)

8.2 Final Comparison Chart

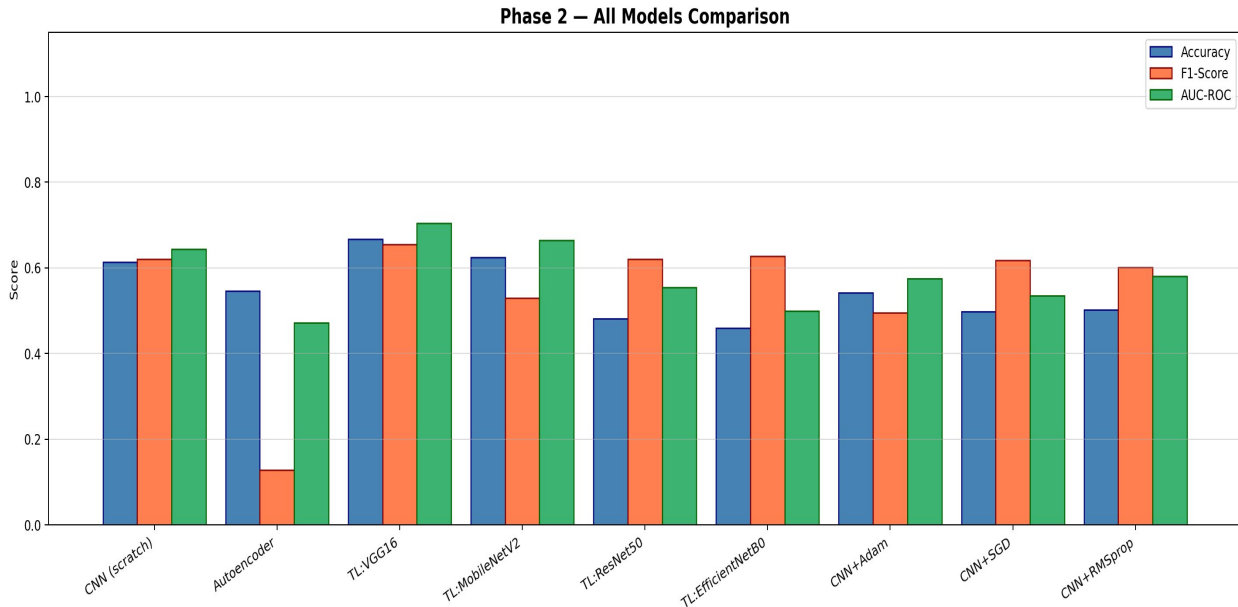


Figure 6: Phase 2 — All Models Comparison. Blue=Accuracy, Orange=F1-Score, Green=AUC-ROC. VGG16 leads overall.

9. Phase 1 vs Phase 2 Comparison

Note: Phase 1 uses 13-class classification (harder task); Phase 2 uses binary classification (Normal vs Diseased). The comparison below shows the progression in model capability and representation learning.

Model	Phase	Task	Accuracy	AUC
Naïve Bayes	1	13-class (HOG+LBP)	0.4313	0.8257*
KNN	1	13-class (HOG+LBP)	0.5063	0.7888*
Gradient Boosting	1	13-class (HOG+LBP)	0.5125	0.8495*
Random Forest	1	13-class (HOG+LBP)	0.5500	0.8567*
CNN+Adam (8ep)	2	Binary (end-to-end)	0.5422	0.5748
CNN from scratch	2	Binary (end-to-end)	0.6133	0.6439
TL:MobileNetV2	2	Binary (TL + FT)	0.6244	0.6651
TL:VGG16	2	Binary (TL + FT)	0.6667	0.7048

* Phase 1 AUC is micro-averaged over 13 classes (one-vs-rest), not directly comparable to binary AUC.

10. Key Findings & Limitations

Finding	Detail
VGG16 is best overall	66.67% accuracy — sequential convolutions align well with X-ray edge/texture patterns. Fine-tuning last 20 layers gives meaningful improvement (val_auc: 0.671→0.717)
CNN from scratch is competitive	61.33% accuracy with 25 epochs, significantly better than Phase 1 (55% at 13 classes, harder task). End-to-end learning clearly superior to HOG+LBP.
Autoencoder fails at anomaly detection	AUC=0.4712 — below random. Normal MSE (0.003) is paradoxically HIGHER than Diseased MSE (0.001). The bottleneck (16×16×128) has too much capacity, allowing reconstruction of all inputs.
Adam outperforms SGD/RMSprop (8 epochs)	Adam val_acc=0.600 vs SGD=0.523 vs RMSprop=0.526. However, SGD/RMSprop have higher test F1, suggesting different precision-recall trade-offs.
ResNet50/EfficientNetB0 underperform	Both struggle with 128px grayscale X-rays. These architectures are designed for high-resolution colour images; input mismatch limits transfer.
Class imbalance handled successfully	Unlike Phase 1 where all minority classes had F1=0, Phase 2 CNN achieves recall=0.69 for Diseased class — class weighting is effective.
CPU-only constraint	No GPU available. 25 epochs of CNN training took 37s/step × 66 steps × 25 = ~17 hours. Transfer learning (4 models × 13 epochs) required significant wall time.

11. Conclusions

Phase 2 demonstrates clear improvements over Phase 1 classical ML baselines. VGG16 Transfer Learning achieves the best performance (66.67% accuracy, AUC=0.705) on binary Normal/Diseased classification. The CNN trained from scratch (61.33%) substantially outperforms all Phase 1 classifiers despite being a simpler binary task. The Autoencoder requires significant redesign — a smaller bottleneck and training for more epochs on a cleaner Normal subset would improve anomaly detection capability.

Phase 3 will apply Vision Transformers (ViT), LSTM over temporal patient data, GAN/VAE for synthetic data generation to address class imbalance, and XAI techniques (Grad-CAM, LIME, SHAP) to explain model decisions — moving towards clinically deployable, interpretable AI-assisted diagnosis.

Appendix: Code Reference

CNN Build & Train

```
cnn_model = build_cnn() # 455,809 params
cnn_model.compile(optimizer=Adam(1e-4), loss='binary_crossentropy',
                  metrics=['accuracy', AUC(name='auc'), Precision(), Recall()])
history = cnn_model.fit(
    aug.flow(X_train, y_train, batch_size=32),
    validation_data=(X_val, y_val), epochs=25,
    class_weight={0:0.921, 1:1.094},
    callbacks=[EarlyStopping('val_auc',patience=6), ReduceLROnPlateau()])
# Result: Acc=0.6133 | F1=0.6201 | AUC=0.6439
```

Autoencoder Anomaly Detection

```
autoencoder.fit(X_train_normal, X_train_normal, ...) # normal only
recon_err = np.mean((X_test - autoencoder.predict(X_test))**2, axis=(1,2,3))
threshold = np.percentile(recon_err[y_test==0], 95)
y_pred_ae = (recon_err > threshold).astype(int)
# Avg MSE Normal:0.003101 | Avg MSE Diseased:0.001458
# AE: Acc=0.5467 | F1=0.1282 | AUC=0.4712
```

Transfer Learning (VGG16 — best model)

```
base = VGG16(weights='imagenet', include_top=False, input_shape=(128,128,3))
base.trainable = False # Phase 1: frozen
# ... train for 8 epochs ...
base.trainable = True
for layer in base.layers[:-20]: layer.trainable = False # Phase 2: fine-tune
model.compile(optimizer=Adam(1e-5), ...)
# VGG16: Test Acc=0.6667 | F1=0.6544 | AUC=0.7048
```

References

- [1] Wang, X. et al. (2017). ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks. CVPR.
- [2] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. Nature, 521, 436–444.
- [3] Simonyan, K. & Zisserman, A. (2015). Very Deep Convolutional Networks (VGG). ICLR.
- [4] Sandler, M. et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR.
- [5] He, K. et al. (2016). Deep Residual Learning for Image Recognition (ResNet). CVPR.
- [6] Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling. ICML.
- [7] Kingma, D. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
- [8] TensorFlow 2.20 Documentation: https://www.tensorflow.org/api_docs